

2 Capabilities

2.1 Introduction

In this section, we report the results of a variety of evaluations our team ran on Claude Opus 4.6 to assess its capabilities in areas such as reasoning, software coding, agentic abilities, mathematics, computer use, and knowledge work. We include some tests from specific domains such as finance and life sciences.

Although this Capabilities section is separate from the safety evaluation sections in this system card, many of the capabilities assessed here have direct relevance to model safety (which is why, for example, some of the specific evaluations are mentioned again in [Section 8](#) where we discuss our Responsible Scaling Policy commitments).

We begin with a summary of selected evaluation results, compared across Opus 4.6 and other models from Anthropic and from other developers. This is followed by individual descriptions of all of the evaluations we ran and the specific methodologies we applied.

For evaluations that were also conducted for Claude Opus 4.5, we have largely retained the descriptions from [its system card](#) in what follows.

2.2 Decontamination

A general problem when running capability evaluations on any large language model is that the answers to certain evaluations appear online. They might thus inadvertently be included in the model's training data, contaminating the model's responses. The model's answers might, in these cases, rely on the memorization of specific examples rather than a completely naïve attempt to answer the question, and might therefore produce inflated evaluation scores.

We take various steps to attempt to decontaminate our evaluations; we refer readers to Section 2.2 in the [Claude Opus 4.5 System Card](#) for a full description. Decontamination is not an exact science—we, and all AI model developers, must continue to improve our methods in this area to ensure that all our evaluations can be accepted at face value.

2.3 Overall results summary

Table 2.3.A summarizes many of the evaluations that we discuss in more detail below.

2.4 SWE-bench (Verified and Multilingual)

SWE-bench (Software Engineering Bench) tests AI models on real-world software engineering tasks.

For the [SWE-bench Verified](#) variant, developed by OpenAI, models are shown 500 problems that have been verified by human engineers to be solvable. We also assessed the model on [SWE-bench Multilingual](#). Here, “multilingual” refers to different programming languages: this variant assesses models on their solutions to 300 problems in 9 different languages.

Claude Opus 4.6 achieves 80.84% on SWE-bench Verified and 77.83% on SWE-bench Multilingual. Our SWE-bench results are averaged over 25 trials, each run with adaptive thinking, max effort, default sampling settings (temperature, top_p), and with the thinking blocks included in the sampling results.

For SWE-bench Verified, we found that the following prompt modification resulted in a score of 81.4%:

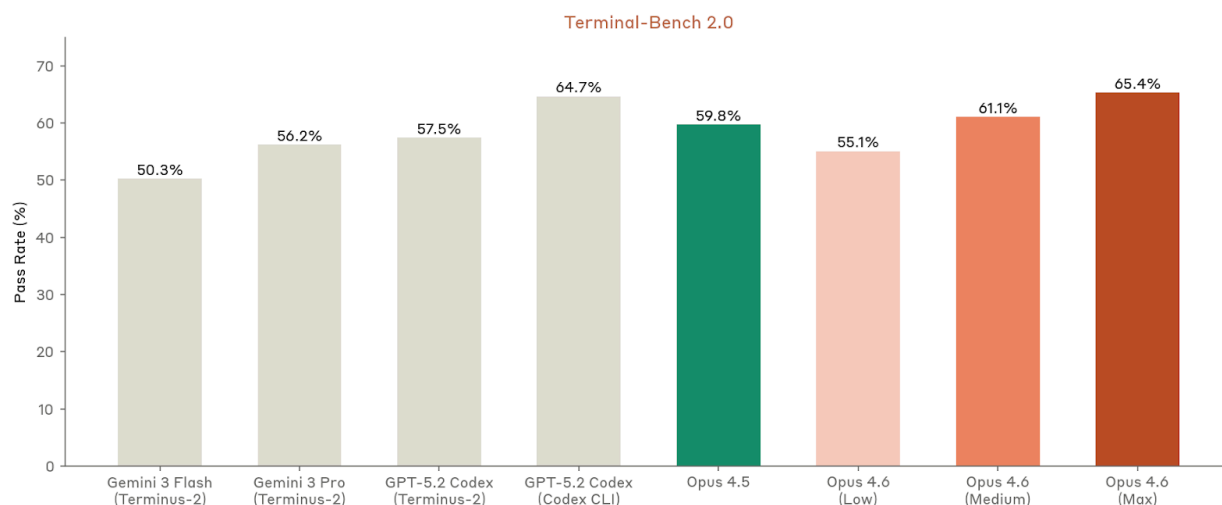
You should use tools as much as possible, ideally more than 100 times. You should also implement your own tests first before attempting the problem. You should take time to explore the codebase and understand the root cause of issues, rather than just fixing surface symptoms. You should be thorough in your reasoning and cover all edge cases.

2.5 Terminal-Bench 2.0

[Terminal-Bench 2.0](#), developed by researchers at Stanford University and the Laude Institute, tests AI models on real-world tasks within terminal or command-line environments.

We ran Terminal-Bench 2.0 in the Harbor scaffold using the Terminus-2 harness with the default parser. All experiments described below, including non-Claude models, ran on a GKE cluster using n2-standard-32 nodes (32 vCPUs, 128 GB RAM, 500 GB persistent disk), in us-central1. We published our adapter open source as a contribution to the Harbor project. Each task runs in an isolated Kubernetes pod; guaranteed resource allocation is set at 1× the benchmark-specified limits, with a hard preemption ceiling at 3×. Timeouts are kept at 1× to keep fidelity to the benchmark specs. Details on this configuration and the rationale behind it are available at anthropic.com/engineering/infrastructure-noise.

Claude Opus 4.6 achieved an average 65.4% pass rate using adaptive thinking at max effort. We ran all 89 tasks 15 times each (1,335 trials), spread across 3 batches at different times to reduce temporal variance.



[Figure 2.5.A] Terminal-Bench 2.0 results. Claude Opus 4.6 achieved a score of 65.4% with max effort. At low effort, Opus 4.6 scores 55.1%, generating 40% fewer output tokens. At medium effort, it scores 61.1%, generating 23% fewer output tokens. For GPT-5.2-Codex, we reproduced 57.5% on Terminus-2 and 64.7% on OpenAI's Codex CLI harness (890 trials). We reproduced 56.2% for Gemini 3 Pro, and 50.3% for Gemini 3 Flash, using the Terminus-2 harness (445 trials).

2.6 OpenRCA

OpenRCA is a root cause analysis benchmark of 335 software failure cases drawn from three real-world enterprise systems (telecom, banking, and online marketplace). It spans 68.5 GB of telemetry across logs, metrics, and traces. Each case requires identifying the root cause of the failure, including the originating component, failure start time, and failure reason. The benchmark was published at ICLR 2025⁴ in the Datasets and Benchmarks track.

⁴ Xu, J., et al. (2025). OpenRCA: Can large language models locate the root cause of software failures? ICLR 2025. <https://openreview.net/forum?id=M4qNlzQYpd>